Basics Day 9 — Session 9 (Hours 33–36)

Functions, Scope, Collections, Modules

Session 9 Overview

Topics Covered Today

- Hour 33: Functions — def, parameters, return
- Hour 34: Scope + common function mistakes
- Hour 35: Functions with collections
- Hour 36: Modules — imports and creating utils.py

Hour 33: Functions — def, parameters, return

Learning Outcomes

- Write a function using `def` with parameters and a body
- Distinguish parameters (the placeholder) from arguments (the value passed)
- Use `return` to send a value back to the caller
- Contrast `print()` (displays to human) with `return` (delivers to program)
- Refactor repeated procedural code into named, reusable functions

Anatomy of a Function

Definition (stored in memory — runs nothing yet)

```
#  keyword  name        parameter
#     |        |             |
def greet_user(name):
    message = f"Hello, {name}!"   # body — indented 4 spaces
    return message                # sends data back to the caller
```

Call (triggers execution)

```
result = greet_user("Alice")   # argument "Alice" -> parameter name
print(result)                  # Hello, Alice!

greet_user("Bob")              # call as many times as needed
greet_user("Charlie")
```

> **Defining** stores the function. **Calling** runs it.

Parameters and Arguments

```python
# One parameter
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")    # "Alice" is the argument
greet("Bob")

# Multiple parameters (matched by position)
def add_two_numbers(a, b):
    total = a + b
    print(f"{a} + {b} = {total}")

add_two_numbers(5, 3)    # a=5, b=3
add_two_numbers(10, 20)  # a=10, b=20
```

- **Parameter** — placeholder variable in the function signature (`name`, `a`, `b`)
- **Argument** — the actual value you supply at call time (`"Alice"`, `5`, `3`)

The return Statement

```python
def multiply(a, b):
    result = a * b
    return result              # sends the value back to the caller

# The call expression is REPLACED by the returned value
answer = multiply(5, 4)        # answer = 20
bigger = answer + 10           # bigger = 30

# return exits immediately — useful for early exit
def safe_divide(a, b):
    if b == 0:
        return "Error: division by zero"
    return a / b               # only reached when b != 0

print(safe_divide(10, 2))      # 5.0
print(safe_divide(10, 0))      # Error: division by zero
```

> Once Python hits `return`, the function stops. Everything after is skipped.

return vs. print()

	`print()`	`return`
Who receives the data?	Human (screen)	The program (caller)
Can the caller use the value?	No	Yes
Function stops after?	No	Yes

```
def bad_sum(a, b):
    print(a + b)            # shows 8 on screen

result = bad_sum(3, 5)      # result is None — not 8!
print(result)               # None

def good_sum(a, b):
    return a + b            # gives 8 to the caller

result = good_sum(3, 5)     # result is 8
print(result)               # 8
```

Demo: Refactoring a Calculator

Before — logic buried in a big menu loop, hard to test

```
if choice == "1":
    result = num1 + num2
    print(f"Result: {result}")
elif choice == "2":
    result = num1 - num2
    print(f"Result: {result}")
```

After — each operation is an isolated, testable function

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def get_float_input(prompt):
```

Lab: Refactor Contact Manager (Hour 33)

Time: 13 minutes

Tasks

Refactor a menu-driven contact manager into named functions:

- `add_contact(contact_list, name)` — appends name; returns nothing
- `list_contacts(contact_list)` — prints all contacts; returns nothing
- `search_contact(contact_list, query)` — **returns** `True` / `False` (no print inside)
- `main()` — contains the `while True` menu loop and calls all three functions

Completion Criteria

✓ Each function does exactly one thing

✓ `search_contact()` uses `return`, not `print()`

Common Pitfalls (Hour 33)

⚠️ **Defining but never calling** — `def greet()` stores the code; you must write `greet()` to run it

⚠️ **Missing colon after `def`** — `def greet(name)` without `:` raises `SyntaxError`

⚠️ **Using `print()` when `return` is needed** — the value looks right on screen, but the caller gets `None`

⚠️ **Confusing parameters and arguments** — `name` in `def greet(name)` is a parameter; `"Alice"` in `greet("Alice")` is the argument

⚠️ **Indentation errors in the body** — all function body lines must be indented exactly 4 spaces

Quick Check (Hour 33)

Q1: What is the difference between `print(result)` and `return result` inside a function?

Q2: In `def add(a, b): return a + b`, what are the parameters and what are the arguments in `add(10, 5)`?

Q3: A function outputs the correct answer on screen, but `result = my_func(3)` gives `None`. What is the bug?

Hour 34: Scope + Common Function Mistakes

Learning Outcomes

- Explain local scope and identify which variables belong to which scope
- Avoid bugs by passing data explicitly instead of relying on global variables
- Diagnose `UnboundLocalError` caused by variable shadowing
- Apply default parameters with correct syntax — required params before defaults
- Recognise the mutable default argument trap with lists and dicts

Local Scope — The Function Bubble

Variables created **inside** a function are **local** — they disappear when the function ends.

```
def make_greeting():
    message = "Good morning!"   # LOCAL variable
    print(message)

make_greeting()     # Good morning!
print(message)      # NameError: name "message" is not defined
```

> What happens inside a function **stays** inside — unless you explicitly `return` it.

Local variables in different functions are completely independent and can share the same name without conflict.

Global Scope and Why Globals Are Risky

```
discount_rate = 0.10   # GLOBAL — readable inside any function

def calculate_price(base_price):
    return base_price - (base_price * discount_rate)   # reads global

print(calculate_price(100))   # 90.0
```

✓ Reading a global is technically allowed.

✗ If `discount_rate` is changed anywhere in 500 lines, every function is silently affected.

✗ Functions with hidden global dependencies are nearly impossible to test in isolation.

> **Best practice:** pass data in through parameters, return data out through `return`.

Variable Shadowing and UnboundLocalError

```
score = 0   # global

def add_points(points):
    score = score + points   # CRASH — UnboundLocalError!
    return score
```

Why it crashes: Python sees `score = ...` and marks ALL uses of `score` in this function as local. Then it tries to read the local `score` on the right side before it has been assigned.

The fix — pass in, return out

```
def add_points(current_score, points):
    return current_score + points   # no globals, no surprises

score = 0
score = add_points(score, 50)
```

The Professional Pattern

```
# BAD — global state creates invisible side-effects
player_health = 100

def take_damage(damage):
    global player_health
    player_health = player_health - damage   # who changed health? hard to track

# GOOD — explicit in, explicit out
def take_damage(current_health, damage):
    return current_health - damage

def heal(current_health, amount):
    return current_health + amount

player_health = 100
player_health = take_damage(player_health, 25)   # 75
player_health = heal(player_health, 10)          # 85
```

Every input is visible in the function call. Every output is captured. No hidden dependencies.

Default Parameters

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice", "Good morning")   # Good morning, Alice!
greet("Bob")                     # Hello, Bob!  <- uses default
greet("Charlie", greeting="Hey") # Hey, Charlie!
```

Critical rule — required parameters FIRST, defaults LAST

```
# WRONG — SyntaxError: non-default argument follows default argument
# def broken(greeting="Hello", name):
#     ...

# CORRECT
def greet(name, greeting="Hello"):
    ...
```

Without this rule Python cannot tell whether `greet("Alice")` fills `greeting` or `name`.

Mutable Default Argument Trap

```
# DANGEROUS — the list [] is created once when the function is defined
def append_item(item, my_list=[]):
    my_list.append(item)
    return my_list

append_item("A")   # ["A"]
append_item("B")   # ["A", "B"]  <- same list persists across calls!
```

Safe pattern — use `None` as the default sentinel

```
def append_item(item, my_list=None):
    if my_list is None:
        my_list = []           # fresh list every call
    my_list.append(item)
    return my_list

append_item("A")   # ["A"]
append_item("B")   # ["B"]  <- independent lists
```

Lab: Calculator Functions (Hour 34)

Time: 14 minutes

Tasks

Build four isolated math functions with no global variables:

- `add(num1, num2)`, `subtract(num1, num2)`, `multiply(num1, num2=1)`, `divide(num1, num2)`
- `divide()` must **return** `"Error: Cannot divide by zero!"` when `num2 == 0` — no crash
- `multiply()` uses a default so `multiply(5)` returns `5`
- A `while True` main loop reads operation choice, converts input to `float`, calls the right function, and prints the result

Completion Criteria

Common Pitfalls (Hour 34)

⚠️ **Accessing a local variable from outside** — `NameError`; fix by returning the value

⚠️ **UnboundLocalError** — triggered by assigning to a name locally while also trying to read it from global scope; fix by passing as a parameter

⚠️ **Using `global` keyword** — almost always a design smell; restructure to pass data in instead

⚠️ **Default param order violated** — required params must precede defaults or `SyntaxError` at definition time

⚠️ **Mutable default (list or dict)** — use `None` sentinel and create the mutable object inside the function body

Quick Check (Hour 34)

Q1: What prints and why?

```
def double(n):
    n = n * 2
x = 5
double(x)
print(x)
```

Q2: Why does `score = score + points` crash with `UnboundLocalError` when `score` is a global variable?

Q3: Write the correct signature for `greet` that has a required `name` and an optional `title` defaulting to `"Ms."`.

Hour 35: Functions with Collections

Learning Outcomes

- Recognise that lists and dicts are passed by reference — the function sees the original, not a copy
- Distinguish mutating a collection in-place from returning a new one
- Choose the right pattern for each context using a clear decision guide
- Write docstrings that declare whether a function mutates or returns (function contracts)
- Apply the filter pattern to extract matching items into a new list

Collections Are Passed by Reference

```
def empty_the_list(my_list):
    my_list.clear()          # modifies the ORIGINAL — no copy was made

groceries = ["Milk", "Eggs", "Bread"]
print(groceries)             # ["Milk", "Eggs", "Bread"]

empty_the_list(groceries)
print(groceries)             # []  <- original was destroyed!
```

> Python passes a **reference** to the collection. Your function and your main code share the **same** list object.

Type	Behaviour when passed to a function
`int`, `str`, `float`, `bool`	Effectively pass by value — safe
`list`, `dict`, `set`	Pass by reference — original can change

Pattern 1: Mutate In-Place

Directly modify the list your function received.

```
def clean_names_inplace(name_list):
    """
    MUTATES the list by cleaning all names in-place.
    The original list is modified. No return value.
    """
    for i in range(len(name_list)):
        name_list[i] = name_list[i].strip().title()

raw = ["  alice smith", "BOB  JONES", " cHarLiE bRoWn "]
clean_names_inplace(raw)
print(raw)   # ["Alice Smith", "Bob Jones", "Charlie Brown"]
```

When to use: sorting existing data, memory-critical operations (millions of items), explicit destructive intent documented in the docstring.

Pattern 2: Return a New Collection

Build and return a fresh list — leave the original untouched.

```
def clean_names_return_new(name_list):
    """
    Returns a NEW list of cleaned, title-cased names.
    Original list is never modified.
    """
    cleaned = []
    for name in name_list:
        cleaned.append(name.strip().title())
    return cleaned

raw = ["  alice smith", "BOB  JONES", " cHarLiE bRoWn "]
result = clean_names_return_new(raw)

print(result)   # ["Alice Smith", "Bob Jones", "Charlie Brown"]
print(raw)      # ["  alice smith", "BOB  JONES", " cHarLiE bRoWn "] <- unchanged
```

When to use: transforming or filtering data, keeping original as a reference, any time safety matters more than memory

Decision Guide + Function Contracts

When to use each pattern

Situation	Recommended pattern
Transform or filter data	Return new collection
Sort or shuffle existing list	Mutate in-place
Keep original as a reference	Return new collection
Memory critical (millions of items)	Mutate in-place
Default choice for beginners	**Return new collection**

Docstrings make the contract explicit

```
def remove_empty_strings(my_list):
    """
    Returns a NEW list with all empty strings removed
```

Demo: Filtering a Collection

```python
scores = [45, 87, 92, 65, 78, 91, 55, 88]

def get_passing_scores(score_list):
    """
    Returns a NEW list containing only scores >= 70.
    Original list is not modified.
    """
    passing = []
    for score in score_list:
        if score >= 70:
            passing.append(score)
    return passing

print("All scores:", scores)
result = get_passing_scores(scores)
print("Passing scores:", result)
# All scores:    [45, 87, 92, 65, 78, 91, 55, 88]
# Passing scores: [87, 92, 78, 91, 88]
```

Lab: Normalize Contacts (Hour 35)

Time: 25 minutes

Task

Write `normalize_contacts(contacts)` that cleans a list of messy contact names:

1. Strip leading and trailing whitespace from each name
2. Convert each name to title case
3. Return a new list (or mutate in-place — declare your choice in the docstring)

```
raw = ["  sarah jones ", "MARK smith", " tina turner  "]
# Expected: ["Sarah Jones", "Mark Smith", "Tina Turner"]
```

Completion Criteria

✓ Function has a docstring that states whether it mutates or returns

Common Pitfalls (Hour 35)

⚠️ **Forgetting to capture the return value** — `normalize(names)` does nothing visible; you need `cleaned = normalize(names)`

⚠️ **Loop variable reassignment** — `for name in contacts: name = name.strip()` reassigns the loop variable, NOT the list item; use `contacts[i] = ...` or build a new list

⚠️ **Assuming a copy was made** — calling `.clear()` or `.append()` inside a function always modifies the original list

⚠️ **Skipping the docstring** — without a mutation statement, future readers cannot tell whether the original data is safe

Quick Check (Hour 35)

Q1: After this runs, what does `scores` contain? Why?

```
def double_all(lst):
    for i in range(len(lst)):
        lst[i] = lst[i] * 2

scores = [1, 2, 3]
double_all(scores)
print(scores)
```

Q2: What is wrong with this mutation attempt?

```
for name in contacts:
    name = name.strip().title()
```

Q3: You need `filter_even(nums)` to return even numbers without changing the original. Which pattern should you use?

Hour 36: Modules — Imports and Creating utils.py

Learning Outcomes

- Import from the standard library using `import`, `from ... import`, and `as` alias forms
- Explain the `__name__` variable and how it controls code execution on import vs. direct run
- Create a custom `utils.py` module with focused, reusable functions
- Import from a custom module using correct file organisation
- Troubleshoot common import errors — naming conflicts, path issues, circular imports

Three Import Forms

Form 1 — Full module import

```
import math
import random

result = math.sqrt(25)          # clear: sqrt comes from math
value = random.randint(1, 10)   # clear: randint comes from random
```

Form 2 — Targeted import

```
from math import sqrt, pi
from random import choice

result = sqrt(25)               # concise; source less obvious
```

Form 3 — Aliased import

The name Special Variable

Every Python file has `__name__`. Its value depends on **how** the file is executed.

Execution method	Value of `__name__`
Run directly (`python utils.py`)	`"__main__"`
Imported (`import utils`)	`"utils"`

Making a file work as both a script and an importable module

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

if __name__ == "__main__":
    # Only runs when the file is executed directly
    # Skipped when the file is imported
```

Naming Best Practices + File Organisation

Avoid shadowing the standard library

```
# If your file is named random.py in the project folder:
import random
random.randint(1, 10)   # AttributeError — your file has no randint!
# Python finds YOUR random.py before the stdlib random module
```

Safe naming conventions

- ✅ `utils.py`, `math_helpers.py`, `validators.py`, `formatters.py`
- ❌ `random.py`, `string.py`, `math.py` — shadow stdlib modules

Standard layout for the Basics course

```
project_folder/
├── main.py      <- user interaction and control flow (orchestrates)
└── utils.py     <- reusable helper functions (provides tools)
```

Demo: Standard Library Imports

```python
import math
import random
from datetime import datetime
from random import choice

# math module
print(math.sqrt(25))          # 5.0
print(math.pi)                # 3.14159...
print(math.ceil(3.2))         # 4

# random module
nums = [1, 2, 3, 4, 5]
print(random.choice(nums))    # random item
print(random.randint(1, 10))  # random int

# datetime
now = datetime.now()
print(f"Year: {now.year}, Month: {now.month}")

# from-import form (no prefix required)
selected = choice(["apple", "banana", "cherry"])
print(selected)
```

Demo: Creating a Custom Module

math_helpers.py

```python
def add(a, b):
    """Add two numbers."""
    return a + b

def power(base, exponent):
    """Raise base to exponent."""
    return base ** exponent

if __name__ == "__main__":
    print(f"add(3, 5) = {add(3, 5)}")
    print(f"power(2, 8) = {power(2, 8)}")
```

use_math_helpers.py (same folder)

```python
from math_helpers import add, power
```

Lab: Build utils.py (Hour 36)

Time: 25 minutes — 4 checkpoints

Checkpoint 1 (5 min): Create `utils.py` and `main.py` in the same folder; add a module docstring to each file

Checkpoint 2 (8 min): Implement `safe_int(prompt)` in `utils.py`

```
def safe_int(prompt):
    """Loops until the user provides a valid integer."""
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("   ERROR: Please enter an integer.")
```

Checkpoint 3 (8 min): Implement `format_money(value)` in `utils.py`

```
def format_money(value):
```

Common Pitfalls (Hour 36)

⚠️ **Naming your file after a stdlib module** — `random.py` in your project shadows `import random`; Python finds your file first

⚠️ **ImportError: cannot import name** — the function does not exist yet in `utils.py`; implement it first, then re-run

⚠️ **No output when running utils.py directly** — you need a `if __name__ == "__main__":` block with test calls

⚠️ **Circular import** — `main.py` imports `utils.py` and `utils.py` imports `main.py`; keep `utils.py` self-contained

⚠️ **Files in different folders** — both `main.py` and `utils.py` must be in the same directory, otherwise `ModuleNotFoundError`

Quick Check (Hour 36)

Q1: Write one example of each import form: `import`, `from ... import`, and `import ... as`.

Q2: A student creates `math.py` in their project folder and then gets `AttributeError: module has no attribute sqrt`. What went wrong?

Q3: What is `__name__` equal to when a Python file is run directly? What is it equal to when the same file is imported?

Session 9 Wrap-Up

What We Covered Today

Hour 33: Functions — def, parameters, return

- `def`, parameters vs. arguments, `return` vs. `print()`
- Refactoring repeated code into clean, testable functions

Hour 34: Scope + Common Mistakes

- Local scope isolates variables; globals create hidden bugs
- Variable shadowing and `UnboundLocalError`; default parameters; mutable default trap

Hour 35: Functions with Collections

- Lists and dicts pass by reference — mutation is real and immediate

Scope Guardrail Reminder

Stay in Basics Scope

✓ `def`, parameters, `return` — core function mechanics
✓ Local and global scope — pass data in, return data out
✓ Default parameters with scalar defaults (`None`, `0`, `""`)
✓ `import`, `from ... import`, `import ... as` — three standard forms
✓ Single-file custom modules (`utils.py`) with `if __name__ == "__main__":`

Not Yet (Advanced Topics)

✗ `*args` and `**kwargs` — variadic arguments
✗ Lambda functions
✗ Decorators
✗ Closures and nested function factories
✗ Packages (`__init__.py`) and relative imports